

# Domande\_ok

## ❓ (1) Operatori di Sobel: Descriverne il funzionamento e illustrarne brevemente le applicazioni.

Gli **operatori di Sobel** sono tra i filtri lineari derivativi più utilizzati per l'**estrazione dei bordi** e l'analisi del gradiente in un'immagine.

Funzionamento

L'operazione di Sobel non consiste in un singolo filtro, ma in una **coppia di maschere (kernel) 3x3** progettate per stimare le derivate parziali dell'intensità luminosa lungo i due assi dell'immagine:

1. **Matrice  $F_x$ :** rileva le variazioni in direzione orizzontale.

$$F_x = \frac{1}{8} \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}$$

2. **Matrice  $F_y$ :** rileva le variazioni in direzione verticale.

$$F_y = \frac{1}{8} \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

- **Differenziazione:** calcola la variazione di intensità nella direzione desiderata.
- **Smoothing (sfocatura):** applica contemporaneamente un filtro di smooth nella direzione ortogonale a quella di derivazione. Questo rende l'operatore molto più **robusto rispetto al rumore** di fondo dell'immagine rispetto a derivate semplici come quelle di Roberts.

Calcolo del Gradiente

Una volta applicati entrambi i filtri, i risultati vengono combinati per ottenere le informazioni complete sul bordo:

- **Modulo del gradiente:** indica la "forza" o nitidezza del bordo in quel punto.
- **Orientazione del gradiente:** indica la direzione di massima variazione. Il bordo dell'oggetto sarà sempre **ortogonale** alla direzione del gradiente.

Applicazioni principali

Le fonti indicano diverse applicazioni fondamentali per questi operatori:

- **Rilevamento dei bordi:** identificazione dei contorni degli oggetti per interpretarne la forma geometrica.
- **Segmentazione:** separazione degli oggetti dallo sfondo basandosi sulle transizioni brusche di luminosità.
- **Base per algoritmi avanzati:** sono il componente fondamentale per metodi più complessi, come il **Canny edge detector**, che utilizza Sobel per il calcolo iniziale del gradiente.
- **Visione Artificiale Industriale:** utilizzati in compiti di misurazione, conteggio e controllo qualità dove è necessario individuare con precisione i profili dei componenti.

## ❓ (2) Operatori di Scharr: Descriverli e illustrarne brevemente le applicazioni.

Gli **operatori di Scharr** sono filtri lineari derivativi (con kernel 3x3) utilizzati nell'elaborazione delle immagini per il calcolo delle derivate parziali e l'estrazione dei bordi.

Funzionamento

Analogamente a Sobel e Prewitt, il metodo di Scharr utilizza una **coppia di maschere** per stimare le variazioni di intensità luminosa lungo gli assi orizzontale e verticale. Questi filtri sono progettati per combinare un'operazione di **differenziazione** con un'operazione di **smoothing** nella direzione ortogonale, al fine di ridurre l'impatto del rumore.

Le matrici (kernel) di Scharr riportate nelle fonti sono:

- **Matrice F<sub>x</sub> (Derivata rispetto a x):** rileva le variazioni orizzontali.

$$F_x = \frac{1}{32} \begin{bmatrix} 3 & 0 & -3 \\ 10 & 0 & -10 \\ 3 & 0 & -3 \end{bmatrix}$$

- **Matrice F<sub>y</sub> (Derivata rispetto a y):** rileva le variazioni verticali.

$$F_y = \frac{1}{32} \begin{bmatrix} 3 & 10 & 3 \\ 0 & 0 & 0 \\ -3 & -10 & -3 \end{bmatrix}$$

Caratteristiche e Applicazioni

La caratteristica distintiva dello Scharr, che lo differenzia dagli altri operatori 3x3, è la sua **migliore invarianza rotazionale**.

Le sue applicazioni principali includono:

- **Stima precisa dell'angolo del gradiente:** Test empirici mostrano che Scharr commette errori significativamente inferiori rispetto a Sobel e Prewitt nella stima dell'orientazione del gradiente. Viene quindi preferito quando la **precisione angolare** è un requisito critico per l'analisi dei bordi.
- **Rilevamento dei bordi ad alta fedeltà:** Viene impiegato per identificare i contorni degli oggetti minimizzando le distorsioni dovute all'orientamento dei bordi stessi rispetto alla griglia dei pixel.
- **Calcolo del Gradiente in OpenCV:** È supportato nativamente dalla funzione **cv.Scharr()**, che restituisce la derivata parziale dell'immagine con un'accuratezza superiore rispetto alla versione standard di Sobel per kernel di piccole dimensioni.

In sintesi, sebbene sia leggermente più complesso del filtro di Prewitt, lo Scharr è lo strumento ideale quando è necessaria la **massima precisione nella direzione del gradiente** tra i filtri locali di base

### 🔗 (3) Operatori di Prewitt: Descriverli e illustrarne le possibili applicazioni.

Gli **operatori di Prewitt** sono filtri lineari derivativi 3x3 utilizzati per l'analisi spaziale delle immagini, in particolare per l'individuazione dei bordi.

Funzionamento

Il funzionamento si basa sull'applicazione di una **coppia di maschere (kernel) di convoluzione** progettate per stimare le derivate parziali dell'intensità dell'immagine lungo le direzioni orizzontale (x) e verticale (y).

Ogni maschera combina un'operazione di **differenziazione** (per rilevare il cambiamento) con un'operazione di **smoothing** o media mobile nella direzione ortogonale (per ridurre l'influenza del rumore). Le matrici specifiche riportate dalle fonti sono:

- **Matrice  $F_x$  (Derivata rispetto a  $x^{**}$ ):**

$$F_x = \frac{1}{8} \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

- **Matrice Fy (Derivata rispetto a y):**

$$F_y = \frac{1}{8} \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$

Dal punto di vista delle prestazioni, il filtro di Prewitt è indicato come **il più efficiente** dal punto di vista computazionale tra i principali filtri derivativi 3x3 (rispetto a Sobel e Scharr).

Possibili Applicazioni

Secondo il materiale didattico, le applicazioni principali includono:

- **Edge Detection (Rilevamento dei bordi):** identificazione dei contorni degli oggetti all'interno di un'immagine.
- **Calcolo del Gradiente:** utilizzo dei risultati di Fx e Fy come componenti vettoriali per determinare il **modulo** (forza della transizione) e l'**orientazione** (direzione ortogonale al bordo) del gradiente di intensità.
- **Segmentazione:** è spesso il primo passo per individuare e separare oggetti di interesse dallo sfondo.
- **Analisi preliminare per il riconoscimento:** estrazione di feature geometriche semplici (come linee e profili) che servono come base per algoritmi di riconoscimento oggetti più complessi.

#### ④ (4) **Canny Edge Detector: Descrivere sinteticamente il funzionamento di questo metodo.**

L'**algoritmo di Canny**, proposto nel 1986, è un metodo multi-stadio progettato per individuare i bordi di un'immagine in modo che siano robusti rispetto al rumore e precisi nella localizzazione.

Il funzionamento si articola in **quattro fasi principali**:

1. **Smooth Gaussiano:** Viene applicata una sfocatura gaussiana per ridurre il rumore e le variazioni di luminosità irrilevanti, che potrebbero generare falsi bordi.
2. **Calcolo del gradiente:** Per ogni pixel si determinano il modulo e l'orientazione del gradiente, individuando le zone di cambiamento rapido di intensità.
3. **Soppressione dei non-massimi:** In questa fase si analizza il modulo del gradiente in direzione ortogonale al bordo. Un pixel viene conservato solo se il suo valore è un

**massimo locale** rispetto ai suoi vicini lungo la direzione del gradiente. L'obiettivo è "assottigliare" i bordi portandoli allo **spessore di un solo pixel**.

4. **Soglia con isteresi:** Per selezionare solo i bordi significativi e garantirne la continuità, si utilizzano **due soglie distinte** ( $T_1$  e  $T_2$ , con  $T_1 > T_2$ ).

-I pixel con valore superiore a  $T_1$  sono considerati subito bordi

- i pixel con valore compreso tra le due soglie sono accettati solo se **direttamente connessi** a un pixel già classificato come bordo forte.

## ❓ (5) **Sharpening: Spiegare cosa si intende generalmente per "sharpening" nell'elaborazione delle immagini.**

Lo **sharpening** è una tecnica di elaborazione delle immagini finalizzata a ottenere un **miglioramento della messa a fuoco** e un **aumento del contrasto locale**. Mentre i filtri di *blur* (come il box filter o il filtro gaussiano) attenuano i dettagli, lo sharpening agisce in modo opposto per rendere i contorni più nitidi.

Funzionamento concettuale e matematico

Il procedimento standard per applicare lo sharpening si articola in tre passaggi chiave:

1. **Sfocatura (Blur):** Si calcola una versione "ammorbidita" dell'immagine originale ( $I$ ) tramite convoluzione con un filtro di smooth ( $F_b$ ), come un filtro gaussiano.
2. **Estrazione dei dettagli ( $M$ ):** Si sottrae l'immagine sfocata dall'originale ( $M = I - I * F_b$ ). Questa operazione permette di evidenziare le zone caratterizzate da **cambiamenti rapidi** di intensità, ovvero i bordi e i dettagli fini.
3. **Riacquisizione del dettaglio:** L'immagine finale ( $I'$ ) si ottiene sommando all'immagine originale i dettagli estratti, moltiplicati per un fattore  $k$ :  $I' = I + k \cdot M$ . Il parametro  $k$  è fondamentale poiché permette di controllare l'intensità dell'effetto di nitidezza desiderato.

Ottimizzazione tramite convoluzione

Grazie alle proprietà algebriche della convoluzione (proprietà distributiva), non è necessario eseguire tutti i passaggi separatamente. È possibile costruire un **singolo filtro equivalente** ( $F_s$ ) che, applicato direttamente all'immagine originale, esegua l'intera operazione di sharpening in un unico passaggio, migliorando l'efficienza computazionale.

In sintesi, lo sharpening "accentua" artificialmente le transizioni di luminosità laddove sono già presenti dei cambiamenti (bordi), dando all'occhio umano la percezione di un'immagine più definita.

## 🔗 (6) Correlazione e Convoluzione

La **correlazione** e la **convoluzione** sono due operazioni fondamentali per l'applicazione di filtri lineari alle immagini. Sebbene siano matematicamente molto simili, presentano differenze algebriche cruciali.

--- Definizioni

Correlazione ( $I \otimes F$ ) È l'operazione in cui il valore di un pixel nell'immagine di uscita è calcolato come la somma dei prodotti tra i coefficienti del filtro ( $F$ ) e i pixel corrispondenti nell'intorno dell'immagine di input ( $I$ ).

La formula è:

$$I'(y, x) = \sum_{i=-d}^d \sum_{j=-d}^d F(i, j) \cdot I(y + i, x + j)$$

Convoluzione ( $I * F$ ) Si ottiene cambiando il segno degli offset nella formula della correlazione. In termini pratici, equivale ad applicare la correlazione dopo aver **ribaltato il filtro sia orizzontalmente che verticalmente** ( $F'(i, j) = F(-i, -j)$ ).

La formula è:

$$I'(y, x) = \sum_{i=-d}^d \sum_{j=-d}^d F(i, j) \cdot I(y - i, x - j)$$

--

Caratteristiche Comuni

Entrambe le operazioni condividono alcune proprietà algebriche fondamentali:

- **Proprietà distributiva rispetto all'addizione:** Applicare un filtro somma di due kernel equivale alla somma dei risultati dei singoli filtri:

$$I * (F + G) = I * F + I * G$$

- **Proprietà associativa rispetto a uno scalare:** Moltiplicare il filtro per una costante scala proporzionalmente il risultato.
- **Equivalenza con filtri simmetrici:** Se il filtro è simmetrico rispetto all'origine (asse verticale e orizzontale), il risultato della correlazione e della convoluzione è identico:

$$I * F = I \otimes F$$

Differenze Principali Le differenze risiedono principalmente nelle proprietà matematiche avanzate e nell'implementazione:

- **Commutatività:** Solo la convoluzione è commutativa ( $I * F = F * I$ ). Nella correlazione, l'ordine dei fattori conta e generalmente:

$$I \otimes F \neq F \otimes I$$

- **Associatività:** Solo la convoluzione è associativa ( $(I * F) * G = I * (F * G)$ ). > [!info] Importanza dell'associatività > Questa proprietà è fondamentale perché permette di combinare più filtri in cascata (come uno smooth e un derivativo) in un unico filtro equivalente prima di applicarlo all'immagine, risparmiando tempo di calcolo.

## ② (7) **Efficienza della Convoluzione: Illustrare la complessità computazionale della convoluzione e descrivere tecniche per migliorarne l'efficienza.**

### 1. Complessità Computazionale Standard

Nell'applicazione di un filtro lineare standard (kernel) di dimensioni  $m \times m$ , il calcolo del valore di ogni singolo pixel dell'immagine risultante richiede:

- $m^2$  **moltiplicazioni** e  $m^2$  **somme**.

Se l'immagine ha dimensioni  $N \times N$ , la complessità totale per l'intera immagine scala con  $O(N^2 \cdot m^2)$ . All'aumentare della dimensione del filtro, il costo computazionale cresce quindi in modo quadratico.

### 2. Tecniche per migliorare l'efficienza

#### I Filtri Separabili

La tecnica di ottimizzazione più rilevante è l'utilizzo di **filtri separabili**. Un filtro si definisce separabile se la sua matrice  $F$  può essere ottenuta come prodotto di un vettore colonna per un vettore riga ( $F = F_y \cdot F_x^T$ ).

- **Funzionamento:** Invece di applicare il kernel  $m \times m$  in un unico passaggio, si applica prima il filtro riga  $1 \times m$  a tutta l'immagine e, sul risultato intermedio ottenuto, si applica il filtro colonna  $m \times 1$ .
- **Vantaggio:** La complessità computazionale scende da  $m^2$  a  $2m$  moltiplicazioni e somme per pixel. Ad esempio, per un filtro  $9 \times 9$ , si passa da 81 operazioni a sole 18 per ogni pixel.

#### Utilizzo delle Proprietà Algebriche

Sfruttando le proprietà della convoluzione, è possibile combinare più passaggi in un'unica operazione:

- **Associatività:** Se dobbiamo applicare in sequenza un filtro di smooth (S) e un filtro derivativo (D), possiamo calcolare preventivamente un unico filtro  $F=S*D$  e applicarlo una sola volta all'immagine ( $I'=I*F$ ).
- **Distributività:** Per operazioni come lo **sharpening**, si può costruire un singolo filtro finale che integri sia l'immagine originale che la componente di contrasto, riducendo il numero di scansioni dell'immagine.

### Implementazioni Specifiche (Box Filter)

Per alcuni filtri particolari esistono algoritmi ancora più efficienti delle implementazioni generiche:

- Il **Box Filter** (media mobile) ha tutti i coefficienti uguali a uno.
- In OpenCV, la funzione **cv.boxFilter** è significativamente più veloce di `cv.filter2D` o `cv.sepFilter2D` poiché utilizza tecniche di ottimizzazione specifiche per questo tipo di kernel. Dai test empirici, `cv.boxFilter` può risultare oltre 15 volte più veloce di una convoluzione standard

## ❓ (8) Operatori di Base (Erosione e Dilatazione): Definirli formalmente e illustrarne brevemente il funzionamento.

### Operatori di Base (Erosione e Dilatazione)

La correlazione e la convoluzione non sono le uniche operazioni fondamentali; nel contesto della morfologia matematica, l'erosione e la dilatazione costituiscono i blocchi base per l'elaborazione delle forme.

#### Erosione ( $F \ominus S$ )

- **Definizione formale:**

$$F \ominus S = \{q \mid S_q \subseteq F\}$$

- **Funzionamento:** L'immagine risultante è composta dall'insieme di tutti i pixel  $q$  tali che, traslando l'elemento strutturante  $S$  in quella posizione, esso risulti interamente contenuto nel foreground ( $F$ ).
- **Effetto visivo:** L'erosione provoca un assottigliamento degli oggetti, eliminando i pixel lungo i bordi e facendo scomparire le componenti connesse che sono più piccole dell'elemento strutturante.



## Dilatazione ( $F \oplus S$ )

- **Definizione formale:**

$$F \oplus S = \{q \mid S_q^r \cap F \neq \emptyset\}$$

- **Funzionamento:** Il risultato è l'insieme di tutti i pixel  $q$  tali che, traslando in essi l'elemento strutturante riflesso ( $S^r$ ), almeno uno dei suoi pixel si sovrapponga (intersezione non vuota) al foreground ( $F$ ) dell'immagine originale.
- **Effetto visivo:** La dilatazione produce un'espansione degli oggetti, aggiungendo pixel lungo i bordi e colmando eventuali piccole lacune o interruzioni nelle forme.

## ❓ (9) Operatore di Apertura: Definirlo formalmente e descriverne il funzionamento.

### Definizione Formale

La formula matematica che lo definisce è:  $F \circ S = (F \ominus S) \oplus S$  dove  $F$  rappresenta l'immagine binaria originale,  $S$  l'elemento strutturante,  $\ominus$  l'operatore di erosione e  $\oplus$  quello di dilatazione.

### Funzionamento e Applicazioni

L'apertura agisce come un filtro morfologico che "pulisce" l'immagine mantenendo le strutture principali. Il suo funzionamento può essere descritto attraverso i seguenti effetti:

- **Rimozione di regioni piccole:** Elimina tutte le componenti connesse o i dettagli di foreground che sono **più piccoli dell'elemento strutturante** utilizzato.
- **Separazione di oggetti:** È molto efficace per **separare oggetti debolmente connessi**, ovvero uniti tra loro solo da istmi o "ponti" sottili di pixel.
- **Smoothing dei contorni:** L'operazione tende a **rimuovere piccole protuberanze** o "spuntoni" lungo i bordi degli oggetti, contribuendo a regolarizzarne la forma.
- **Eliminazione del rumore:** Viene spesso utilizzata dopo la binarizzazione per eliminare artefatti e punti isolati dovuti al rumore che potrebbero essersi fusi con le componenti principali

## ❗ **Hit-or-Miss Transform:** Spiegare cos'è e a cosa può servire.

La **Hit-or-Miss Transform** è un operatore fondamentale della **morfologia matematica** utilizzato specificamente per la ricerca di **configurazioni (pattern) particolari** di pixel all'interno di un'immagine binaria.

A differenza delle operazioni di base come l'erosione o la dilatazione che si concentrano sulla forma del foreground, la Hit-or-Miss esamina contemporaneamente la disposizione dei pixel di **foreground** e di **background**. Per farlo, utilizza un elemento strutturante "composto" da due parti disgiunte:

- La prima parte deve trovare una corrispondenza nel **foreground** (Hit).
- La seconda parte deve trovare una corrispondenza nel **background**, ovvero nel complemento dell'immagine (Miss).

A cosa può servire?

Secondo quanto riportato nei temi d'esame, questa trasformata trova applicazione in diversi ambiti dell'analisi d'immagine:

- **Riconoscimento di pattern geometrici:** È utile per individuare strutture specifiche come angoli, punti isolati, intersezioni o particolari forme geometriche in immagini binarie.
- **Estrazione di caratteristiche topologiche:** Viene impiegata per identificare elementi come le estremità di una linea o giunzioni tra segmenti.
- **Base per algoritmi complessi:** Costituisce l'operazione elementare su cui si basano processi più avanzati, come l'**assottigliamento (thinning)** per estrarre lo scheletro mediano di un oggetto, o l'ispessimento (thickening)

### ❓ (10) **Rimozione di Componenti:** Spiegare come rimuovere piccole componenti connesse usando la morfologia.

Per rimuovere piccole componenti connesse utilizzando la **morfologia matematica**, l'operatore fondamentale da impiegare è l'**apertura** ( $F \circ S$ ).

Il funzionamento di questa tecnica si basa sulla combinazione sequenziale di due operazioni:

1. **Erosione:** È il primo passo del processo. Si applica un'erosione all'immagine utilizzando un **elemento strutturante**  $S$  le cui dimensioni siano leggermente superiori a quelle delle componenti che si desidera eliminare. Durante questa fase, le componenti connesse o i dettagli di foreground che sono **più piccoli dell'elemento strutturante scompaiono completamente**, mentre le componenti più grandi vengono "assottigliate".
2. **Dilatazione:** Si applica quindi una dilatazione sul risultato dell'erosione, utilizzando lo stesso elemento strutturante  $S$ . Lo scopo è quello di riportare le componenti sopravvissute alla loro dimensione originale. Poiché le piccole componenti sono state

rimosse totalmente durante l'erosione, la dilatazione non ha alcun "seme" da cui farle ricrescere, lasciando così l'immagine pulita.

## Applicazioni principali

Questa tecnica viene spesso utilizzata dopo la binarizzazione per:

- **Eliminare il rumore:** Rimuovere piccoli punti isolati o artefatti generati da una sogliaatura imperfetta.
- **Separare oggetti:** Scollegare componenti che sono unite solo da istmi o "ponti" molto sottili di pixel.
- **Smoothing:** Regularizzare i contorni eliminando piccole protuberanze

## ❓ (11) Chiusura di "Buchi": Descrivere come chiudere o riempire piccoli buchi in un'immagine (binaria o meno).

Per chiudere o riempire piccoli buchi all'interno delle componenti di un'immagine, i documenti suggeriscono due approcci principali basati sulla **morfologia matematica** e sull'**analisi delle componenti connesse**.

### 1. Operatore di Chiusura ( $F \cdot S$ )

L'operatore di **chiusura** è lo strumento morfologico standard per risolvere questo problema.

- **Definizione:** È definito formalmente come una **dilatazione seguita da un'erosione**, eseguite con lo stesso elemento strutturante  $S$ :  $F \cdot S = (F \oplus S) \ominus S$ .
- **Funzionamento:** La dilatazione iniziale espande i bordi del foreground, facendo sì che i pixel aggiunti vadano a "tappare" i piccoli buchi o le lacune interne. L'erosione successiva riporta le componenti alla loro dimensione originale, ma non può ripristinare i buchi che sono stati colmati durante la fase precedente.
- **Limitazioni:** Questa tecnica è efficace per buchi che sono **più piccoli dell'elemento strutturante** utilizzato; la dimensione di quest'ultimo va quindi scelta con attenzione in base alla dimensione dei buchi da eliminare.

### 2. Riempimento tramite Analisi del Background

Per buchi di dimensioni maggiori o quando si desidera un controllo più preciso basato sull'area, si può utilizzare l'algoritmo di **etichettatura delle componenti connesse** applicato al "negativo" dell'immagine.

- **Logica:** Si estraggono le componenti connesse dei pixel di **background** (i pixel a 0).

- **Identificazione dei buchi:** Si assume solitamente che la componente di background con l'**area maggiore** sia l'effettivo sfondo dell'immagine, mentre tutte le altre componenti isolate di background siano "buchi" all'interno del foreground.
- **Azione:** Tutti i pixel appartenenti a queste componenti di background minori vengono impostati al valore del foreground (es. 255), eliminando così i buchi in modo definitivo

## ② (12) **Etichettatura (Labeling):** Descrivere il funzionamento dell'algoritmo di etichettatura delle componenti connesse.

- ▶ Procedura molto importante nell'analisi di immagini binarie:
  - Individuare automaticamente le diverse componenti connesse in un'immagine, assegnando loro etichette (tipicamente numeriche)
- ▶ Algoritmo comunemente utilizzato:
  1. Si scorre l'immagine e, per ogni pixel di foreground **p**, si considerano i pixel di foreground vicini già visitati:
    - se nessuno è etichettato, si assegna una nuova etichetta a **p**;
    - se uno è etichettato, si assegna a **p** la stessa etichetta;
    - se più di uno è etichettato, si assegna a **p** una delle etichette e si annotano le equivalenze.
  2. Si definisce un'unica etichetta per ogni insieme di etichette marcate come equivalenti e si effettua una seconda scansione assegnando le etichette finali

## ② (13) **Definizioni di Topologia:** Definire formalmente i termini "vicini", "percorso" e "componente connessa".

- ▶ I vicini di un pixel **p** sono quelli aventi distanza unitaria da **p**
- ▶ Un percorso di lunghezza  $n$  da **p** a **q** è una sequenza di pixel  $\{p_0 = p, p_1, \dots, p_n = q\}$  tale che, in accordo con la metrica scelta,  $p_i$  è un vicino di  $p_{i+1}$ ,  $0 \leq i < n$
- ▶ Una componente connessa è un sottoinsieme di  $F$  (o di  $F^*$ ) tale che, presi due qualsiasi dei suoi pixel, esiste tra questi un percorso appartenente a  $F$  (o di  $F^*$ ). A seconda della metrica adottata si parla di 4-connessione o di 8-connessione

## ② (14) **Trasformata Distanza: Descrivere sinteticamente il funzionamento e indicare alcune possibili applicazioni.**

La **Trasformata Distanza** è un'operazione fondamentale nell'analisi di immagini binarie, composte da un insieme di pixel di foreground (F) e un insieme di background (F\*).

### Funzionamento

Il principio di base consiste nel generare un'immagine in cui ogni pixel del foreground viene etichettato con il valore della sua **distanza minima dal pixel di background più vicino**.

Per definire questa distanza, si adottano solitamente metriche discrete come:

- **Distanza d4 (City-block):** calcolata considerando solo i movimenti orizzontali e verticali.
- **Distanza d8 (Chessboard):** calcolata considerando anche i movimenti diagonali.

Un algoritmo comune per implementarla in modo efficiente prevede **due scansioni sequenziali** dell'immagine: una **diretta** (dall'alto verso il basso e da sinistra a destra) per propagare le distanze dai vicini superiori e sinistri, e una **inversa** (dal basso verso l'alto e da destra a sinistra) per aggiornare i valori con i contributi dei vicini inferiori e destri. In ambiente Python, OpenCV mette a disposizione la funzione `cv.distanceTransform()` per questo scopo.

### Applicazioni

Le fonti elencano diverse applicazioni pratiche della trasformata distanza:

- **Misurazione geometrica:** calcolo accurato di lunghezze e spessori degli oggetti all'interno dell'immagine.
- **Estrazione dello scheletro:** poiché la trasformata produce massimi locali di intensità lungo gli assi mediani delle figure, viene usata per algoritmi di **assottigliamento (thinning)**, utili per rappresentare la forma essenziale di oggetti allungati.
- **Template matching:** impiegata come tecnica per misurare il grado di somiglianza tra un modello noto e porzioni dell'immagine.
- **Robotica:** utilizzata per la pianificazione di percorsi, permettendo ai sistemi autonomi di individuare direzioni di movimento ottimali o di aggirare ostacoli.

## ② (15) **Spazio Colore RGB: Descriverlo e illustrarne vantaggi e svantaggi.**

Il modello RGB (Red, Green, Blue) è uno spazio colore **additivo** basato sulla tricromia: i tre colori primari spettrali vengono sommati in diverse proporzioni (con valori da 0 a 255 a 8-bit) per generare l'intera gamma cromatica. Geometricamente è rappresentabile come un cubo, dove l'origine (0, 0, 0) corrisponde al nero e il vertice opposto (255, 255, 255) al bianco. La combinazione dei tre canali al massimo valore produce la luce bianca, simulando il funzionamento della retina umana (coni) e dei dispositivi digitali.

- **Vantaggi:** È lo standard nativo per l'hardware digitale (sensori delle fotocamere, monitor, proiettori). Ha una corrispondenza diretta 1:1 con i pixel dello schermo, rendendo l'elaborazione e la visualizzazione computazionalmente efficienti e prive di conversioni matematiche complesse.
- **Svantaggi:** Non è intuitivo per l'essere umano (è difficile stimare i valori RGB esatti di un "marrone chiaro"). Inoltre, i canali sono **altamente correlati**: se si cambia la luminosità di un pixel, variano tutti e tre i valori R, G e B, modificando inevitabilmente anche la percezione del colore stesso (cromatismi).

### ❓ (16) Spazi Colore HSL/HSV: Descriverli e illustrarne vantaggi e svantaggi.

Gli spazi HSL (Hue, Saturation, Lightness) e HSV (Hue, Saturation, Value) nascono per descrivere il colore in modo più vicino alla percezione psicologica umana.

Geometricamente si rappresentano come un cilindro o un cono.

1. **Hue (Tonalità):** L'angolo del cerchio cromatico ( $0^{\circ}$ - $360^{\circ}$ ) che definisce il colore puro (es. rosso, verde).
  2. **Saturation (Saturazione):** La purezza del colore (da grigio neutro a colore vivido).
  3. **Lightness/Value (Luminosità):** La quantità di luce. HSL mappa il bianco puro all'estremità superiore, HSV lo mappa insieme ai colori saturi, sfumando verso il nero in basso.
- **Vantaggi:** Estremamente intuitivi per l'uomo (per schiarire un colore basta agire su L/V, senza toccare la tonalità). Sono ideali nella computer vision per la **segmentazione del colore** (isolare un oggetto rosso è facile impostando un intervallo sul canale Hue, indipendentemente dalle ombre).
  - **Svantaggi:** Presentano singolarità matematiche (es. se la saturazione è zero, la tonalità è indefinita). Richiedono una conversione matematica onerosa da/verso lo spazio RGB prima della visualizzazione a schermo e non sono lineari rispetto alla reale sensibilità dell'occhio umano.



### ② (17) **Istogramma:** Definire cos'è l'istogramma di un'immagine digitale e a cosa può servire.

L'istogramma è un grafico statistico che rappresenta la **distribuzione delle intensità luminose** all'interno di un'immagine digitale. Sull'asse delle ascisse ( $X$ ) sono riportati i livelli di grigio (da 0 per il nero a 255 per il bianco), mentre sull'asse delle ordinate ( $Y$ ) è indicato il numero di pixel (frequenza) che possiedono quella specifica intensità. In un'immagine a colori, si possono calcolare istogrammi separati per i canali R, G e B o un istogramma globale della luminanza.

- **A cosa serve:**

- **Analisi dell'esposizione:** Permette di capire subito se un'immagine è sottoesposta (istogramma schiacciato a sinistra), sovraesposta (schiacciato a destra) o a basso contrasto (concentrato al centro).
- **Soglia (Thresholding):** Identifica i picchi per separare l'oggetto dallo sfondo (es. binarizzazione del testo).
- **Miglioramento dell'immagine:** Fornisce i dati di partenza fondamentali per applicare algoritmi correttivi automatici, come il contrast stretching o l'equalizzazione.

### ② (18) **Contrast Stretching:** Descrivere il funzionamento di questa tecnica.

Il Contrast Stretching è una tecnica di elaborazione puntuale delle immagini utilizzata per migliorare il contrasto di foto "piatte" o sbiadite. Spesso, a causa di una cattiva illuminazione, l'istogramma occupa solo una frazione ristretta dell'intervallo dinamico disponibile (es. pixel compresi solo tra i valori 60 e 140 su una scala da 0 a 255). Questa tecnica "stira" linearmente questo intervallo ridotto per fargli occupare l'intera scala disponibile.

Il funzionamento si basa sull'individuazione del valore minimo ( $I_{min}$ ) e massimo ( $I_{max}$ ) effettivamente presenti nell'immagine. Ogni pixel originale  $I(x, y)$  viene rimappato nel nuovo valore  $I_{new}(x, y)$  tramite una trasformazione lineare:

$$I_{new} = \frac{I - I_{min}}{I_{max} - I_{min}} \cdot 255$$

In questo modo, il pixel più scuro ( $I_{min}$ ) diventa 0 (nero puro), il più chiaro ( $I_{max}$ ) diventa 255 (bianco puro), e i valori intermedi vengono distribuiti proporzionalmente, aumentando la separazione tonale e la nitidezza visiva dell'immagine.

② (19) **Trasformazioni Affini e Mapping Diretto:** Spiegare quali problemi presenta l'applicazione di una trasformazione affine tramite mapping diretto e come risolverli.

Le trasformazioni affini (rotazione, scalatura, traslazione) modificano la geometria di un'immagine tramite una matrice. Nel **mapping diretto**, il computer cicla sui pixel dell'immagine di origine  $(x, y)$ , applica la matrice e calcola le nuove coordinate  $(x', y')$  nell'immagine di destinazione. Essendo le immagini digitali griglie discrete di pixel interi, questo approccio matematico diretto genera gravi problemi geometrici.

- **Problemi principali:**

- **Buchi (Holes):** I calcoli generano coordinate decimali. Arrotondandole all'intero più vicino, in caso di ingrandimento o rotazione, molti pixel della griglia di destinazione non vengono mai colpiti, creando un effetto "groviera" (punti neri vuoti).
- **Sovrapposizioni (Overlaps):** Più pixel di origine possono collidere sullo stesso pixel di destinazione, sovrascrivendosi e perdendo dettagli.
- **Soluzione:** Si utilizza il **Mapping Inverso** combinato all'**Interpolazione**. Il computer cicla sulla griglia vuota di *destinazione*  $(x', y')$ , applica la trasformazione *inversa* per trovare il punto di origine  $(x, y)$  e ne calcola il colore stimandolo tramite la media dei pixel circostanti (interpolazione bilineare o bicubica), azzerando buchi e artefatti.

② (20) **Template Matching:** Spiegare in cosa consiste e, nello specifico, definirne la variante "rigida" discutendone pregi e difetti.

Il **Template Matching** è una tecnica di visione artificiale utilizzata per individuare le posizioni di un'immagine che corrispondono a un modello di riferimento, chiamato **template**.

In cosa consiste

Invece di confrontare intere immagini tra loro, si definiscono uno o più pattern modello (piccole immagini del particolare da cercare). L'algoritmo fa scorrere questo template sull'immagine sorgente, misurando in ogni possibile posizione il grado di **somiglianza** (matching) tra il template e la porzione di immagine sottostante.

Nelle applicazioni più semplici si confrontano direttamente i valori dei pixel, ma per una maggiore robustezza si possono utilizzare altre caratteristiche come i **bordi** o l'**orientazione del gradiente**. In OpenCV, questa operazione viene eseguita tramite la funzione `cv.matchTemplate()`, che supporta diversi metodi di calcolo come la somma delle differenze al quadrato (SQDIFF) o la correlazione (CCORR).



## Il Template Matching "Rigido"

La variante "**rigida**" (menzionata esplicitamente come domanda d'esame) è quella in cui il template viene cercato eseguendo esclusivamente operazioni di **traslazione**. In questa configurazione, l'algoritmo assume che l'oggetto da ricercare nella scena abbia la **stessa dimensione, orientamento e forma** del modello originale.

### Pregi

- **Semplicità ed efficacia:** È molto più utile e robusto del confronto pixel-a-pixel di intere immagini, poiché permette di localizzare l'oggetto ovunque esso si trovi nella scena (risolve il problema della traslazione).
- **Facilità di implementazione:** È supportato nativamente dalle principali librerie di visione artificiale.

### Difetti

- **Sensibilità alle variazioni geometriche:** Fallisce se l'oggetto nella scena è **ruotato**, ha una **scala diversa** (è più grande o più piccolo) o è visto da una **prospettiva differente** rispetto al template.
- **Sensibilità alle deformazioni:** Non è in grado di gestire oggetti che cambiano forma o che presentano variabilità intrinseca (ad esempio diverse espressioni facciali o pose di un corpo).
- **Sensibilità all'illuminazione:** Cambiamenti nella luce ambientale o riflessi possono alterare significativamente i valori dei pixel, compromettendo la misura di somiglianza.
- **Costo computazionale:** Far scorrere il template in ogni pixel dell'immagine può essere oneroso, specialmente per template grandi o immagini ad alta risoluzione.

## ❓ (21) **Confronto Pixel-a-Pixel: Spiegare perché generalmente non è efficace per cercare un oggetto all'interno di un'immagine in applicazioni reali.**

il confronto diretto pixel-a-pixel tra due immagini raramente è utile nelle applicazioni reali per diverse ragioni fondamentali:

- **Differenze geometriche:** Anche piccoli cambiamenti di **traslazione, rotazione, scala e prospettiva** tra l'oggetto nel modello e l'oggetto nella scena rendono i valori dei pixel non corrispondenti.
- **Variabilità degli oggetti:** Gli oggetti possono subire **deformazioni** o presentare variazioni intrinseche (ad esempio, diverse razze di cani o diverse espressioni facciali) che cambiano la disposizione dei pixel.

- **Condizioni di illuminazione:** Cambiamenti nella luce ambientale o riflessi alterano drasticamente i valori numerici dei pixel, anche se l'oggetto è lo stesso.
- **Fattori tecnici:** La presenza di **rumore** nelle immagini e l'utilizzo di tecniche di acquisizione differenti possono influenzare il risultato del confronto

## ❓ (22) **Algoritmo Mean-Shift:** Descrivere il funzionamento di questo algoritmo.

Nel contesto della visione artificiale, viene applicato principalmente per il **tracking (inseguimento) di oggetti** all'interno di sequenze video.

Funzionamento nel tracking di oggetti

L'applicazione del Mean-Shift per l'inseguimento segue una logica specifica descritta nelle fonti:

1. **Definizione della regione iniziale:** Si considera una regione  $W$  (finestra di ricerca) che nel frame precedente conteneva l'oggetto da inseguire.
2. **Creazione della mappa di confidenza ( $C$ ):** Per ogni pixel del nuovo frame, si calcola la probabilità che esso appartenga all'oggetto. Questa mappa viene solitamente generata analizzando caratteristiche come il colore o la luminosità.
  - Un metodo comune consiste nel calcolare l'**istogramma** dei valori dell'oggetto nel frame precedente: la confidenza di un pixel nel nuovo frame sarà data dalla frequenza del suo valore all'interno di tale istogramma (procedura nota come *Back-Projection*).
3. **Spostamento iterativo:** Ad ogni iterazione, l'algoritmo calcola la **media pesata** della mappa di confidenza  $C$  all'interno della finestra  $W$ . Successivamente, la finestra  $W$  viene **spostata** verso il centro di massa calcolato.
4. **Convergenza:** Il processo si ripete finché la media non converge (ovvero la finestra smette di spostarsi in modo significativo) o finché non viene raggiunto un numero massimo di iterazioni prestabilito.

Implementazione in OpenCV

In ambiente Python, l'algoritmo è disponibile tramite la funzione **cv.meanShift()**. Per generare la mappa di confidenza necessaria all'algoritmo, si utilizza tipicamente **cv.calcBackProject()**, che proietta l'istogramma del modello dell'oggetto sui nuovi frame per evidenziare le aree con caratteristiche simili.

## ❓ (23) **Modello Pinhole Camera:** Definire cosa sono i parametri estrinseci in questo modello

## Parametri Estrinseci

I parametri estrinseci definiscono la posizione e l'orientamento della telecamera nello spazio tridimensionale rispetto al sistema di coordinate della scena (mondo).

- **Funzione:** Trasformano le coordinate di un punto dal sistema di riferimento del "mondo" ( $P_w$ ) a quello della "camera" ( $P_c$ ).
- **Componenti:** Consistono in una **matrice di rotazione** ( $R$ ) e un **vettore di traslazione** ( $t$ ).
- **Gradi di libertà:** Sono in totale **6 parametri**: 3 per la rotazione (spesso descritti tramite gli angoli di Eulero) e 3 per la traslazione.
- **Applicazione:** Permettono di descrivere il movimento della camera attorno a una scena statica o lo spostamento di un oggetto rispetto a una camera fissa.

## Parametri Intrinseci

I parametri intrinseci sono specifici dell'hardware della telecamera e del suo obiettivo; descrivono come i punti già convertiti nel sistema di riferimento della camera vengono proiettati sul piano immagine per diventare pixel.

- **Lunghezza focale ( $f$ ):** Una caratteristica ottica dell'obiettivo che determina l'ingrandimento dell'immagine.
- **Punto principale ( $c_x, c_y$ ):** Rappresenta le coordinate del punto in cui l'asse ottico interseca il piano immagine.
- **Fattori di scala ( $s_x, s_y$ ):** Definiscono la relazione tra le unità di misura della scena 3D e la dimensione dei singoli pixel sul sensore.
- **Distorsione radiale ( $k_1, k_2, k_3, \dots$ ):** Parametri che modellano le imperfezioni della lente, che causano spostamenti dei punti rispetto al modello geometrico ideale (come la distorsione a barilotto o a cuscinetto)